E-COMMERCE DATA ANALYST PORTFOLIO

15 Real-World PostgreSQL Queries Based on Dashboard Metrics

This portfolio document contains 15 functional PostgreSQL queries mapped directly to the business metrics displayed on an e-commerce performance dashboard. These queries demonstrate foundational to advanced SQL competencies, including aggregation, joins, conditional logic, CTEs, and window functions, optimized for PostgreSQL syntax.

High-Level Business Metrics

1. How do you calculate the total revenue and total number of delivered orders?

Demonstrates basic aggregation and filtering.

```
SELECT
    SUM(revenue) AS total_revenue,
    COUNT(order_id) AS total_delivered_orders
FROM orders
WHERE order_status = 'Delivered';
```

2. How do you find the Average Order Value (AOV)?

Uses `::numeric` casting, which is a PostgreSQL best practice when performing division to avoid integer truncation, and rounds to two decimal places.

```
SELECT
    ROUND((SUM(revenue) / COUNT(DISTINCT order_id))::numeric, 2) AS
avg_order_value
FROM orders
WHERE order_status = 'Delivered';
```

3. What is the overall return rate for the store?

Highlights conditional aggregation using `CASE WHEN` and explicitly handles type casting for accurate percentage calculation in Postgres.

```
SELECT
    ROUND(
        (COUNT(CASE WHEN order_status = 'Returned' THEN 1 END)::numeric /
        COUNT(order_id)) * 100,
    2) AS return_rate_percentage
FROM orders;
```

Customer Insights & Segmentation

4. How many active customers do we have compared to churned customers?

Demonstrates standard grouping to output comparative counts.

```sql
SELECT
    customer_status,
    COUNT(customer_id) AS total_customers
FROM customers
GROUP BY customer_status;
```

5. Can you break down the total customer base by membership tier?

Validates ability to segment categorical data and sort results logically.

```sql
SELECT
    membership_tier,
    COUNT(customer_id) AS customer_count
FROM customers
GROUP BY membership_tier
ORDER BY customer_count DESC;
```

6. How much revenue is generated by subscribed versus non-subscribed customers?

Requires an `INNER JOIN` between the orders and customers tables to link transactional data with user attributes.

```sql
SELECT
    c.subscription_status,
    SUM(o.revenue) AS total_revenue
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
WHERE o.order_status = 'Delivered'
GROUP BY c.subscription_status;
```

7. How do you calculate the percentage of revenue from repeated buyers vs. single-time buyers?

Utilizes a Common Table Expression (CTE) to pre-aggregate order counts per user, a strong intermediate SQL technique. Note the use of `GROUP BY 1` which is highly idiomatic in PostgreSQL.

```
WITH CustomerOrderCounts AS (
    SELECT
        customer_id,
        COUNT(order_id) AS order_count
    FROM orders
    GROUP BY customer_id
)
SELECT
    CASE WHEN order_count > 1 THEN 'Repeated Buyer' ELSE 'Single Time Buyer' END
AS buyer_type,
    SUM(o.revenue) AS total_revenue
FROM CustomerOrderCounts c
JOIN orders o ON c.customer_id = o.customer_id
GROUP BY 1;
```

Product & Category Performance

8. What are the top 5 product categories by total revenue?

Combines joins, grouping, and ordering with a `LIMIT` clause to identify top drivers.

```
SELECT
    p.category_name,
    SUM(o.revenue) AS total_revenue
FROM orders o
JOIN products p ON o.product_id = p.product_id
WHERE o.order_status = 'Delivered'
GROUP BY p.category_name
ORDER BY total_revenue DESC
LIMIT 5;
```

9. Can you identify the top 5 individual products driving the most revenue?

A straightforward ranking query essential for inventory and SKU performance analysis.

```sql
SELECT
    p.product_name,
    SUM(o.revenue) AS total_revenue
FROM orders o
JOIN products p ON o.product_id = p.product_id
GROUP BY p.product_name
ORDER BY total_revenue DESC
LIMIT 5;
```

Sales Channels & Demographics

10. Which acquisition channels bring in the most active customers?

Highlights filtering by a specific condition (Active) alongside grouping by channel.

```sql
SELECT
    acquisition_channel,
    COUNT(customer_id) AS active_customer_count
FROM customers
WHERE customer_status = 'Active'
GROUP BY acquisition_channel
ORDER BY active_customer_count DESC;
```

11. How is revenue distributed across different devices (Mobile, Desktop, Tablet)?

A core query for informing UI/UX and cross-platform marketing decisions.

```sql
SELECT
    device_used,
    SUM(revenue) AS total_revenue
FROM orders
WHERE order_status = 'Delivered'
GROUP BY device_used;
```

12. What is the total revenue broken down by customer gender?

Demonstrates demographic segmentation by joining user metadata with transactional records.

```sql
SELECT
    c.gender,
    SUM(o.revenue) AS total_revenue
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
WHERE o.order_status = 'Delivered'
GROUP BY c.gender;
```

Trends & Advanced Analysis

13. How do you extract the quarterly sales trend?

Uses PostgreSQL's specific `TO_CHAR()` function to elegantly format dates into readable quarters (e.g., 'Q1 2024') while ensuring proper chronological sorting.

```sql
SELECT
    TO_CHAR(order_date, '"Q"Q YYYY') AS quarter_year,
    COUNT(order_id) AS total_orders
FROM orders
GROUP BY TO_CHAR(order_date, '"Q"Q YYYY')
ORDER BY MIN(order_date);
```

14. Can you identify our key accounts (top 5 customers by revenue)?

Highlights user-level aggregation for VIP account management and discount tracking.

```sql
SELECT
    customer_id,
    SUM(revenue) AS total_revenue,
    SUM(discount_given) AS total_discount
FROM orders
GROUP BY customer_id
ORDER BY total_revenue DESC
LIMIT 5;
```

15. How would you calculate Year-over-Year (YoY) revenue growth?

A highly sought-after analytical skill. This query leverages a CTE alongside PostgreSQL Window Functions (`LAG()`) to compute current vs previous year revenue and outputs a precise growth percentage.

```sql
WITH YearlyRevenue AS (
    SELECT
        EXTRACT(YEAR FROM order_date) AS sales_year,
        SUM(revenue) AS current_revenue
    FROM orders
    GROUP BY 1
)
SELECT
    sales_year,
    current_revenue,
    LAG(current_revenue) OVER (ORDER BY sales_year) AS previous_year_revenue,
    ROUND(
        ((current_revenue - LAG(current_revenue) OVER (ORDER BY sales_year)) /
        LAG(current_revenue) OVER (ORDER BY sales_year)::numeric) * 100,
    2) AS yoy_growth_percentage
FROM YearlyRevenue;
```